File Integrity Monitoring System

Complete Documentation & User Guide

Version: 1.0
Date: April 2026
Author: Development Team

Table of Contents

- 1. Overview
- 2. Key Features
- 3. System Architecture
- 4. Installation & Setup
- 5. Configuration Guide
- 6. Usage Instructions
- 7. Live Sample Demo Output
- 8. API Reference
- 9. Troubleshooting
- 10. Best Practices

1. Overview

The File Integrity Monitoring System is a Python-based utility that tracks and monitors the integrity of files in a specified directory. It uses cryptographic hashing (SHA-256) to detect any unauthorized or accidental changes to files, creating a secure audit trail of file modifications. This tool is essential for security, compliance, and data integrity verification.

The system creates a baseline of all files by computing their SHA-256 hash values and storing them in a JSON database. On subsequent runs, it compares current file hashes against the baseline to identify: newly created files, modified files, deleted files, and unchanged files.

2. Key Features

Feature	Description
SHA-256 Hashing	Cryptographically secure file hashing to detect any changes
Baseline Creation	Automatic creation of initial hash database for comparison
Change Detection	Identifies modified, new, and deleted files instantly
JSON Storage	Human-readable JSON format for easy integration and manual inspection
PDF Reporting	Automatic PDF report generation for audit trails and documentation

3. System Architecture

3.1 Core Components

- • calculate_hash(): Computes SHA-256 hash of files in 4KB chunks
- • scan_files(): Recursively scans directory and returns file-hash dictionary
- • load_previous_hashes(): Retrieves stored hash baseline from JSON
- • save_hashes(): Persists current hashes to JSON for future comparison
- • compare_hashes(): Compares old vs new hashes and generates report
- • generate_pdf(): Creates formatted PDF report with findings

3.2 Data Flow Diagram

Scan Directory → Calculate Hashes → Load Previous Hashes → Compare → Generate Report → Save New Hashes

3.3 File Storage Structure

- file_hashes.json: Stores SHA-256 hash values for all files
- file_integrity_report.pdf: Generated PDF report with audit findings

4. Installation & Setup

4.1 Requirements

- Python 3.6 or higher
- reportlab library (for PDF generation)
- hashlib (built-in Python module)
- json (built-in Python module)
- datetime (built-in Python module)
- os (built-in Python module)

4.2 Installation Steps

1. Install reportlab library:

```
pip install reportlab
```

2. Save the Python script to your project directory:

```
file_integrity_monitor.py
```

3. Verify installation by running:

```
python file_integrity_monitor.py
```

5. Configuration Guide

5.1 Configuration Variables

Variable	Description & Default Value
MONITOR_DIR	Directory to monitor. Default: "." (current directory)
HASH_FILE	JSON file for storing hashes. Default: "file_hashes.json"
REPORT_PDF_FILE	PDF report filename. Default: "file_integrity_report.pdf"

5.2 Customization Example

To monitor a different directory, modify the script:

```
MONITOR_DIR = "/path/to/directory"
HASH_FILE = "custom_hashes.json"
REPORT_PDF_FILE = "custom_report.pdf"
```

6. Usage Instructions

6.1 Basic Usage

Run the script to monitor files:

```
python file_integrity_monitor.py
```

6.2 Workflow Scenarios

Scenario 1: First Run (Baseline Creation)

1. On the first execution, the system detects no previous hash file and creates a baseline. This establishes the initial state of all files in the directory.

Scenario 2: Subsequent Runs (Change Detection)

2. On subsequent runs, the system compares current hashes against the stored baseline and reports any modifications, new files, or deleted files.

Scenario 3: Automated Monitoring

3. Use cron jobs (Linux/Mac) or Task Scheduler (Windows) to run the script periodically. This creates an automated audit trail of all file changes.

6.3 Interpreting Console Output

- • [OK] Unchanged: File has not been modified
- • [MODIFIED]: File hash differs from baseline (file was changed)
- • [NEW]: File did not exist in previous baseline
- • [DELETED]: File existed in baseline but is no longer present

7. Live Sample Demo Output

7.1 Scenario A: Initial Baseline Creation

First run - no previous data exists:

```
Console Output:
Scanning files...

No previous record found. Creating baseline...
Baseline created successfully.

=== File Integrity Report ===

Baseline created. No changes to report yet.
✅ PDF report saved: file_integrity_report.pdf
```

Generated file_hashes.json contains hashes for all files in the directory.

7.2 Scenario B: Detecting Changes (Mixed Modifications)

Second run - files have been modified, added, and deleted:

```
Console Output:
Scanning files...

=== File Integrity Report ===

[OK] Unchanged: ./readme.txt | Hash: a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6
[OK] Unchanged: ./config.ini | Hash: b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7
[MODIFIED] ./main.py | Old Hash: c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8 | New Hash:
d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9
[NEW] ./backup_2026_04_04.zip | Hash: e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0
[DELETED] ./old_script.py | Last Known Hash: f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1

⚠ Changes detected in monitored files.
✅ PDF report saved: file_integrity_report.pdf
```

Key Observations:

- • 2 files remained unchanged (readme.txt, config.ini)
- • 1 file was modified (main.py)
- • 1 new file was added (backup_2026_04_04.zip)
- • 1 file was deleted (old_script.py)

7.3 Scenario C: All Files Intact (No Changes)

Third run - no changes since last check:

```
Console Output:
Scanning files...

=== File Integrity Report ===

[OK] Unchanged: ./readme.txt | Hash: a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6
[OK] Unchanged: ./config.ini | Hash: b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7
[OK] Unchanged: ./main.py | Hash: d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9
[OK] Unchanged: ./backup_2026_04_04.zip | Hash:
e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0

✔ All files are intact. No changes detected.
📄 PDF report saved: file_integrity_report.pdf
```

This output indicates a stable system where no files have been added, removed, or modified since the last verification.

8. API Reference

8.1 Function: calculate_hash(file_path)

Purpose: Calculates the SHA-256 hash of a file

- Parameters:
  - file_path (str): Full path to the file
- Returns: SHA-256 hash string (or None if error)

Efficiency: Reads file in 4KB chunks to handle large files

8.2 Function: scan_files(directory)

Purpose: Recursively scans directory and computes hashes

- Parameters:
  - directory (str): Path to directory to scan
- Returns: Dictionary {file_path: hash_value}

8.3 Function: load_previous_hashes()

Purpose: Loads stored hashes from JSON file

Parameters: None

- Returns: Dictionary of previous hashes (or empty dict if file doesn't exist)

8.4 Function: save_hashes(hashes)

Purpose: Saves hash dictionary to JSON file

- Parameters:
  - hashes (dict): Dictionary {file_path: hash_value}

Returns: None (saves to file_hashes.json)

8.5 Function: compare_hashes(old_hashes, new_hashes)

Purpose: Compares hashes and generates report

- Parameters:
  - old_hashes (dict): Previous hash baseline
  - new_hashes (dict): Current file hashes

Returns: List of report lines with change summary

8.6 Function: generate_pdf(title, content_lines, filename)

Purpose: Generates formatted PDF report

- Parameters:
  - title (str): Report title
  - content_lines (list): List of content strings
  - filename (str): Output PDF filename

Returns: None (creates PDF file)

9. Troubleshooting

Issue: ModuleNotFoundError: No module named 'reportlab'

Solution: Install reportlab with: pip install reportlab

Issue: Permission Denied error when accessing files

Solution: Run script with appropriate permissions. On Linux/Mac, use: sudo python file_integrity_monitor.py

Issue: No previous record found (baseline not created)

Solution: This is normal on first run. The script automatically creates a baseline.

Issue: FileNotFoundError for file_hashes.json

Solution: This occurs on first run. The system creates this file automatically.

Issue: Changes detected unexpectedly

Solution: Check if files were modified. Some programs auto-save or update files. Run again to update baseline if changes are intentional.

Issue: PDF not generated

Solution: Ensure reportlab is installed and check disk space. Verify write permissions in the directory.

Issue: Very slow performance on large directories

Solution: The system reads files in 4KB chunks. This is normal for large datasets. Consider monitoring subdirectories separately.

Issue: JSON file corruption

Solution: Delete file_hashes.json and run the script again to recreate baseline.

10. Best Practices

1. Regular Monitoring Schedule

4. Set up automated scans using cron (Linux/Mac) or Task Scheduler (Windows) to run daily or weekly for continuous monitoring.

2. Archive Reports

5. Maintain historical copies of PDF reports for audit trail purposes. Store them in a secure location separate from monitored files.

3. Exclude Temporary Files

6. Consider excluding temporary directories (.tmp, cache, etc.) from monitoring to reduce noise and improve performance.

4. Backup Hash Database

7. Keep backup copies of file_hashes.json in secure locations to prevent loss of baseline data if the original is compromised.

5. Review Changes Promptly

8. When [MODIFIED] or [NEW] files are detected, review them immediately to ensure authorized changes. Investigate unexpected modifications.

6. Version Control Integration

9. Use alongside version control systems (Git) to track code changes while using this tool for monitoring binary files and assets.

7. Monitor Critical Directories

10. Prioritize monitoring critical system directories, configuration files, and sensitive data folders.

8. Secure the Hash Database

11. Protect file_hashes.json with proper file permissions to prevent unauthorized modification or deletion of the baseline.

Additional Resources

- SHA-256 Hashing: Cryptographically secure one-way hash function
- JSON Format: Human-readable data interchange format
- ReportLab: Python PDF generation library
- Use Cases: System security, compliance auditing, data integrity verification

End of Documentation